

transformer-classification

November 7, 2025

1 Inisiasi Program

1.1 Instalasi dan Impor Library

```
[22]: !pip install tensorflow  
      !pip install wordcloud  
      !pip install imblearn  
      !pip install seaborn  
      !pip install transformers datasets
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)  
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)  
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)  
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)  
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.6.0)  
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)  
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)  
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)  
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.0)  
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.5)  
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)  
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)  
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)  
Requirement already satisfied: termcolor>=1.1.0 in
```

/usr/local/lib/python3.12/dist-packages (from tensorflow) (3.2.0)
 Requirement already satisfied: typing-extensions>=3.6.6 in
 /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
 Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-
 packages (from tensorflow) (2.0.0)
 Requirement already satisfied: grpcio<2.0,>=1.24.3 in
 /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.76.0)
 Requirement already satisfied: tensorboard~=2.19.0 in
 /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
 Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-
 packages (from tensorflow) (3.10.0)
 Requirement already satisfied: numpy<2.2.0,>=1.26.0 in
 /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
 Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-
 packages (from tensorflow) (3.15.1)
 Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in
 /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.3)
 Requirement already satisfied: wheel<1.0,>=0.23.0 in
 /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow)
 (0.45.1)
 Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
 (from keras>=3.5.0->tensorflow) (13.9.4)
 Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages
 (from keras>=3.5.0->tensorflow) (0.1.0)
 Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages
 (from keras>=3.5.0->tensorflow) (0.17.0)
 Requirement already satisfied: charset_normalizer<4,>=2 in
 /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
 (3.4.4)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
 packages (from requests<3,>=2.21.0->tensorflow) (3.11)
 Requirement already satisfied: urllib3<3,>=1.21.1 in
 /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
 (2.5.0)
 Requirement already satisfied: certifi>=2017.4.17 in
 /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
 (2025.10.5)
 Requirement already satisfied: markdown>=2.6.8 in
 /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow)
 (3.10)
 Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in
 /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow)
 (0.7.2)
 Requirement already satisfied: werkzeug>=1.0.1 in
 /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow)
 (3.1.3)
 Requirement already satisfied: MarkupSafe>=2.1.1 in
 /usr/local/lib/python3.12/dist-packages (from

werkzeug>=1.0.1->tensorboard~=2.19.0->tensorflow) (3.0.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow)
(4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow)
(2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-
packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow) (0.1.2)
Requirement already satisfied: wordcloud in /usr/local/lib/python3.12/dist-
packages (1.9.4)
Requirement already satisfied: numpy>=1.6.1 in /usr/local/lib/python3.12/dist-
packages (from wordcloud) (2.0.2)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages
(from wordcloud) (11.3.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-
packages (from wordcloud) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (1.3.3)
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-
packages (from matplotlib->wordcloud) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (1.4.9)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->wordcloud)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib->wordcloud) (1.17.0)
Requirement already satisfied: imblearn in /usr/local/lib/python3.12/dist-
packages (0.0)
Requirement already satisfied: imbalanced-learn in
/usr/local/lib/python3.12/dist-packages (from imblearn) (0.14.0)
Requirement already satisfied: numpy<3,>=1.25.2 in
/usr/local/lib/python3.12/dist-packages (from imbalanced-learn->imblearn)
(2.0.2)
Requirement already satisfied: scipy<2,>=1.11.4 in
/usr/local/lib/python3.12/dist-packages (from imbalanced-learn->imblearn)
(1.16.3)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in
/usr/local/lib/python3.12/dist-packages (from imbalanced-learn->imblearn)
(1.6.1)
Requirement already satisfied: joblib<2,>=1.2.0 in

/usr/local/lib/python3.12/dist-packages (from imbalanced-learn->imblearn)
 (1.5.2)
 Requirement already satisfied: threadpoolctl<4,>=2.0.0 in
 /usr/local/lib/python3.12/dist-packages (from imbalanced-learn->imblearn)
 (3.6.0)
 Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-
 packages (0.13.2)
 Requirement already satisfied: numpy!=1.24.0,>=1.20 in
 /usr/local/lib/python3.12/dist-packages (from seaborn) (2.0.2)
 Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-
 packages (from seaborn) (2.2.2)
 Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in
 /usr/local/lib/python3.12/dist-packages (from seaborn) (3.10.0)
 Requirement already satisfied: contourpy>=1.0.1 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
 (1.3.3)
 Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-
 packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
 (4.60.1)
 Requirement already satisfied: kiwisolver>=1.3.1 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
 (1.4.9)
 Requirement already satisfied: packaging>=20.0 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
 (25.0)
 Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-
 packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
 Requirement already satisfied: pyparsing>=2.3.1 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
 (3.2.5)
 Requirement already satisfied: python-dateutil>=2.7 in
 /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
 (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-
 packages (from pandas>=1.2->seaborn) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-
 packages (from pandas>=1.2->seaborn) (2025.2)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
 packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
 Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-
 packages (4.57.1)
 Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-
 packages (4.0.0)
 Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
 packages (from transformers) (3.20.0)
 Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in

/usr/local/lib/python3.12/dist-packages (from transformers) (0.36.0)
 Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
 Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
 Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
 Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2024.11.6)
 Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
 Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.1)
 Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.6.2)
 Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
 Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
 Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
 Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
 Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
 Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
 Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2025.3.0)
 Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (3.13.2)
 Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transformers) (4.15.0)
 Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->transformers) (1.2.0)
 Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.4.4)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.11)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.5.0)
 Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2025.10.5)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2.6.1)

Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.4.0)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (25.4.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.8.0)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (6.7.0)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (0.4.1)

Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.22.0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)

```
[31]: # Library dasar
from wordcloud import WordCloud
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
import re

# Library untuk preprocessing data
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.text import Tokenizer
from sklearn.utils.class_weight import compute_class_weight
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import RandomOverSampler
```

```

# Library untuk pretrained model
from transformers import BertTokenizer, TFBertForSequenceClassification

# Library untuk model building
from tensorflow.keras.layers import (
    Input, Embedding, LSTM, Dense, Layer, MultiHeadAttention,
    LayerNormalization, Dropout, Add, GlobalAveragePooling1D
)
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping

# Library untuk evaluasi model
from sklearn.metrics import classification_report, confusion_matrix

```

1.2 Import Dataset

```

[5]: # Nama file yang sudah di-upload
nama_file = 'Data.csv'

# Membaca file CSV dengan encoding yang berbeda
try:
    df = pd.read_csv(nama_file, encoding='utf-8')
except UnicodeDecodeError:
    print("UTF-8 decoding failed, trying latin-1...\n")
    df = pd.read_csv(nama_file, encoding='latin-1')

# Menampilkan informasi umum
print("Informasi Umum Dataset:")
df.info()

```

UTF-8 decoding failed, trying latin-1...

Informasi Umum Dataset:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23644 entries, 0 to 23643
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Tweet       23644 non-null  object
1   sentiment   23644 non-null  int64
dtypes: int64(1), object(1)
memory usage: 369.6+ KB

```

1.3 Impor Pre-trained Model

```
[24]: # Tentukan model yang akan digunakan (IndoBERT base, case-unspecific)
model_name = "indobenchmark/indobert-base-p1"

# 1. Muat Tokenizer
tokenizer = BertTokenizer.from_pretrained(model_name)

# 2. Muat Model (TF = TensorFlow)
# Tentukan jumlah label (positif/negatif = 2)
model = TFBertForSequenceClassification.from_pretrained(model_name,
↳ num_labels=2)
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94:

UserWarning:

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(

tokenizer_config.json: 0%| | 0.00/2.00 [00:00<?, ?B/s]

vocab.txt: 0.00B [00:00, ?B/s]

special_tokens_map.json: 0%| | 0.00/112 [00:00<?, ?B/s]

config.json: 0.00B [00:00, ?B/s]

tf_model.h5: 0%| | 0.00/656M [00:00<?, ?B/s]

TensorFlow and JAX classes are deprecated and will be removed in Transformers v5. We recommend migrating to PyTorch classes or pinning your version of Transformers.

All model checkpoint layers were used when initializing TFBertForSequenceClassification.

Some layers of TFBertForSequenceClassification were not initialized from the model checkpoint at indobenchmark/indobert-base-p1 and are newly initialized: ['classifier']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

2 Praproses Data

2.1 Filter dan Rename Data Records

```
[6]: # Hanya ambil sentimen positif dan negatif (tidak mengambil netral)
df = df[df['sentiment'].isin([1, 2]).reset_index(drop=True)

# Rename sentimen angka menjadi teks
df['sentiment'] = df['sentiment'].map({1: 'Positif', 2: 'Negatif'})

# Tampilkan contoh dataset yang telah diubah
df.head()
```

```
[6]:
```

	Tweet sentiment
0	Ketahui informasi pembagian #PPKM di wilayah J... Positif
1	Tempat Ibadah di Wilayah PPKM Level 1 Boleh Be... Positif
2	Juru bicara Satgas Covid-19, Wiku Adisasmito m... Positif
3	Ketahui informasi pembagian #PPKM di wilayah J... Positif
4	Kementerian Agama menerbitkan Surat Edaran Nom... Positif

2.2 Visualisasi Distribusi Kata pada Raw Data

```
[9]: # Gabungkan semua teks mentah menjadi satu string besar
raw_text = " ".join(df['Tweet'].astype(str))

# Gunakan wordcloud untuk visualisasi distribusi kata
wordcloud = WordCloud(width=800, height=400, background_color='white').
    generate(raw_text)

# Plot wordcloud
plt.figure(figsize=(12, 8))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud dari Data Mentah')
plt.show()
```

Word Cloud dari Data Mentah



2.3 Data Cleaning

2.3.1 Stopword List

Berikut adalah list dari stopwords yang kurang akan berguna dalam data training, sehingga sebaiknya dibersihkan dengan cara dihapus dari dataset.

```
[10]: stopwords_list = [
    'ada', 'adalah', 'adanya', 'adapun', 'agak', 'agaknya', 'agar', 'akan',
    'akankah', 'akhir', 'akhiri', 'akhirnya', 'aku', 'akulah', 'amat',
    'amatlah', 'anda', 'andalah', 'antar', 'antara', 'antaranya', 'apa',
    'apaan', 'apabila', 'apakah', 'apalagi', 'atas', 'atau', 'ataukah',
    'ataupun', 'awal', 'awalnya', 'bagaimana', 'bagaimanakah', 'bagaimanapun',
    'bagi', 'bagian', 'bahkan', 'bahwa',
    'bahwasanya', 'baik', 'bakal', 'bakalan', 'balik', 'banyak', 'bapak',
    'baru', 'bawah', 'beberapa', 'begini', 'beginian', 'beginikah', 'beginilah',
    'begitu', 'begitukah', 'begitulah', 'begitupun', 'bekerja', 'belakang',
    'belakangan', 'belum', 'belumlah', 'benar', 'benarkah', 'benarlah',
    'berada', 'berakhir', 'berakhirilah', 'berakhirnya', 'berapa', 'berapakah',
    'berapalah', 'berapapun', 'berarti',
    'berawal', 'berbagai', 'berdatangan', 'beri', 'berikan', 'berikut',
    'berikutnya', 'berjumlah', 'berkali-kali', 'berkata', 'berkehendak',
    'berkeinginan', 'berkenaan', 'berlainan', 'berlalu', 'berlangsung',
    'berlebihan', 'bermacam', 'bermacam-macam', 'bermaksud', 'bermula',
    'bersama', 'bersama-sama', 'bersiap', 'bersiap-siap', 'bertanya',
    'bertanya-tanya', 'berturut',
```

'berturut-turut', 'bertutur', 'berujar', 'berupa', 'besar', 'betul',
 ⇨ 'betulkah', 'biasa', 'biasanya', 'bila', 'bilakah', 'bisa', 'bisakah',
 ⇨ 'boleh', 'bolehkah', 'bolehlah', 'buat', 'bukanlah', 'bukannya', 'bulan',
 ⇨ 'bung', 'cara', 'caranya', 'cukup', 'cukupkah', 'cukuplah', 'cuma',
 ⇨ 'dahulu', 'dalam', 'dan', 'dapat', 'dari', 'daripada', 'datang', 'dekat',
 ⇨ 'demi', 'demikian', 'demikianlah', 'dengan',
 'depan', 'di', 'dia', 'diakhiri', 'diakhirinya', 'dialah', 'diantara',
 ⇨ 'diantaranya', 'diberi', 'diberikan', 'diberikannya', 'dibuat', 'dibuatnya',
 ⇨ 'didapat', 'didatangkan', 'digunakan', 'diibaratkan', 'diibaratkannya',
 ⇨ 'diingat', 'diingatkan', 'diinginkan', 'dijawab', 'dijelaskan',
 ⇨ 'dijelaskannya', 'dikarenakan', 'dikatakan', 'dikatakannya', 'dikerjakan',
 ⇨ 'diketahui', 'diketahuinya', 'dikiranya',
 'dilakukan', 'dilalui', 'dilihat', 'dimaksud', 'dimaksudkan',
 ⇨ 'dimaksudkannya', 'dimaksudnya', 'diminta', 'dimintai', 'dimisalkan',
 ⇨ 'dimulai', 'dimulailah', 'dimulainya', 'dimungkinkan', 'dini', 'dipastikan',
 ⇨ 'diperbuat', 'diperbuatnya', 'dipergunakan', 'diperkirakan',
 ⇨ 'diperlihatkan', 'diperlukan', 'diperlukannya', 'dipersoalkan',
 ⇨ 'dipertanyakan', 'dipunyai', 'diri', 'dirinya',
 'disampaikan', 'disebut', 'disebutkan', 'disebutkannya', 'disini',
 ⇨ 'disinilah', 'ditambahkan', 'ditandaskan', 'ditanya', 'ditanyai',
 ⇨ 'ditanyakan', 'ditegaskan', 'ditujukan', 'ditunjuk', 'ditunjuki',
 ⇨ 'ditunjukkan', 'ditunjukkannya', 'dituntun', 'diturunkan', 'diturunkan',
 ⇨ 'dituturkan', 'dituturkannya', 'diucapkan', 'diucapkannya', 'diungkapkan',
 ⇨ 'dong', 'dua', 'dulu', 'enggak',
 'engkaulah', 'entah', 'entahlah', 'guna', 'gunakan', 'hal', 'hampir',
 ⇨ 'hanya', 'hanyalah', 'hari', 'harus', 'haruslah', 'harusnya', 'hendak',
 ⇨ 'hendaklah', 'hendaknya', 'hingga', 'ia', 'ialah', 'ibarat', 'ibaratnya',
 ⇨ 'ibu', 'ikut', 'ingat', 'ingat-ingat', 'ingin', 'inginkah', 'inginkan',
 ⇨ 'ini', 'inikah', 'inilah', 'itu', 'itukah', 'itulah', 'jadi', 'jadilah',
 ⇨ 'jadinya', 'jauh', 'jawab', 'jawaban',
 'jawabnya', 'jelas', 'jelaskan', 'jelaslah', 'jelasnya', 'jika', 'jikalau',
 ⇨ 'juga', 'jumlah', 'jumlahnya', 'justru', 'kala', 'kalau', 'kalaulah',
 ⇨ 'kalaupun', 'kali', 'kalian', 'kami', 'kamilah', 'kamu', 'kamulah', 'kan',
 ⇨ 'kapan', 'kapankah', 'kapanpun', 'karena', 'karenanya', 'kasus', 'kata',
 ⇨ 'katakan', 'katakanlah', 'katanya', 'ke', 'keadaan', 'kebetulan', 'kecil',
 ⇨ 'kedua', 'keduanya',
 'keinginan', 'kelak', 'kelima', 'kelihatan', 'kelihatannya', 'keliru',
 ⇨ 'kembali', 'kemudian', 'kemungkinan', 'kemungkinannya', 'kenapa', 'kepada',
 ⇨ 'kepadanya', 'kesejahteraan', 'keseluruhan', 'keseluruhannya',
 ⇨ 'keterlaluhan', 'ketika', 'khususnya', 'kini', 'kinilah', 'kira',
 ⇨ 'kira-kira', 'kiranya', 'kita', 'kitalah', 'kok', 'kurang', 'lagi',
 ⇨ 'lagian', 'lah', 'lain', 'lainnya', 'lalu',

'lama', 'lamanya', 'lanjut', 'lanjutnya', 'lebih', 'lewat', 'lima', 'luar',
 ↳ 'macam', 'maka', 'makanya', 'makin', 'malah', 'malahan', 'mampu',
 ↳ 'mampukah', 'mana', 'manakala', 'manalagi', 'masa', 'masalah', 'masalahnya',
 ↳ 'masih', 'masihkah', 'masing', 'masing-masing', 'mau', 'maupun',
 ↳ 'melainkan', 'melakukan', 'melalui', 'melihat', 'melihatnya', 'memang',
 ↳ 'memastikan', 'memberi',
 'memberikan', 'membuat', 'memerlukan', 'memihak', 'meminta', 'memintakan',
 ↳ 'memisalkan', 'memperbuat', 'mempergunakan', 'memperkirakan',
 ↳ 'memperlihatkan', 'mempersiapkan', 'mempersoalkan', 'mempertanyakan',
 ↳ 'mempunyai', 'memulai', 'menjadi', 'menuju', 'menurut', 'menyeluruh',
 ↳ 'menyiapkan', 'merasa', 'mereka', 'merekalah', 'merupakan', 'meski',
 ↳ 'meskipun', 'meyakini', 'meyakinkan', 'minta',
 'mirip', 'misal', 'misalkan', 'misalnya', 'mula', 'mulai', 'mulailah',
 ↳ 'mulanya', 'mungkin', 'mungkinkah', 'nah', 'naik', 'namun', 'nantinya',
 ↳ 'nantinya', 'nyaris', 'nyatanya', 'oleh', 'olehnya', 'pada', 'padahal',
 ↳ 'padanya', 'pak', 'paling', 'panjang', 'pantas', 'para', 'pasti',
 ↳ 'pastilah', 'penting', 'pentingnya', 'per', 'percuma', 'perlu', 'perlukah',
 ↳ 'perlunya', 'pernah', 'persoalan',
 'pertama', 'pertama-tama', 'pertanyaan', 'pertanyakan', 'pihak',
 ↳ 'pihaknya', 'pukul', 'pula', 'pun', 'punya', 'rasa', 'rasanya', 'rata',
 ↳ 'rupanya', 'saat', 'saatnya', 'saja', 'sajalah', 'saling', 'sama',
 ↳ 'sama-sama', 'sambil', 'sampai', 'sampai-sampai', 'sana', 'sangat',
 ↳ 'sangatlah', 'satu', 'saya', 'sayalah', 'se', 'sebab', 'sebabnya',
 ↳ 'sebagai', 'sebagaimana', 'sebagainya', 'sebagian',
 'sebaik', 'sebaik-baiknya', 'sebaiknya', 'sebaliknya', 'sebanyak',
 ↳ 'sebelum', 'sebelumnya', 'sebenarnya', 'seberapa', 'sebesar', 'sebetulnya',
 ↳ 'sebisanya', 'sebuah', 'sebut', 'sebutlah', 'sebutnya', 'secara',
 ↳ 'secukupnya', 'sedang', 'sedangkan', 'sedemikian', 'sedikit', 'sedikitnya',
 ↳ 'seenaknya', 'segala', 'segalanya', 'segera', 'seharusnya', 'sehingga',
 ↳ 'seingat', 'sejak',
 'sejauh', 'sejenak', 'sejumlah', 'sekadar', 'sekadarnya', 'sekali',
 ↳ 'sekali-kali', 'sekalian', 'sekaligus', 'sekalipun', 'sekarang',
 ↳ 'sekaranglah', 'sekecil', 'seketika', 'sekiranya', 'sekitar', 'sekitarnya',
 ↳ 'sela', 'selain', 'selaku', 'selalu', 'selama', 'selama-lamanya',
 ↳ 'selamanya', 'selanjutnya', 'seluruh', 'seluruhnya', 'semacam', 'semakin',
 ↳ 'semampu', 'semampunya', 'semasa',
 'semasih', 'semata', 'semata-mata', 'semaunya', 'sementara', 'semisal',
 ↳ 'semisalnya', 'sempat', 'semua', 'semuanya', 'semula', 'sendiri',
 ↳ 'sendirian', 'sendirinya', 'seolah', 'seolah-olah', 'seorang', 'sepanjang',
 ↳ 'sepantasnya', 'sepantasnyalah', 'seperlunya', 'seperti', 'sepertinya',
 ↳ 'sepihak', 'sering', 'seringnya', 'serta', 'serupa', 'sesaat', 'sesampai',
 ↳ 'sesegera', 'sesekali',

```

    'seseorang', 'sesuatu', 'sesuatunya', 'sesudah', 'sesudahnya', 'setelah',
    ↪ 'setempat', 'setengah', 'seterusnya', 'setiap', 'setidaknya', 'setinggi',
    ↪ 'seusai', 'sewaktu', 'siap', 'siapa', 'siapakah', 'siapapun', 'sini',
    ↪ 'sinilah', 'suatu', 'sudah', 'sudahkah', 'sudahlah', 'supaya', 'tadi',
    ↪ 'tadinya', 'tahu', 'tahun', 'tak', 'tanpa', 'tanya', 'tanyakan', 'tanyanya',
    ↪ 'tapi', 'tegas', 'tegasnya',
    'telah', 'tempat', 'tengah', 'tentang', 'tentu', 'tentulah', 'tentunya',
    ↪ 'tepat', 'terakhir', 'terasa', 'terbanyak', 'terdahulu', 'terdapat',
    ↪ 'terdiri', 'terhadap', 'terhadapnya', 'teringat', 'teringat-ingat',
    ↪ 'terjadi', 'terjadilah', 'terjadinya', 'terkira', 'terlalu', 'terlebih',
    ↪ 'terlihat', 'termasuk', 'ternyata', 'tersebut', 'tersebutlah', 'tertentu',
    ↪ 'tertuju', 'terus', 'terutama',
    'tetap', 'tetapi', 'tiap', 'tiba', 'tiba-tiba', 'tidaklah', 'tidaknya',
    ↪ 'tiga', 'tinggi', 'toh', 'tunjuk', 'turut', 'tutur', 'tuturnya', 'ucap',
    ↪ 'ucapnya', 'ujar', 'ujarnya', 'umumnya', 'ungkap', 'ungkapnya', 'untuk',
    ↪ 'usah', 'usai', 'wah', 'wahai', 'waktu', 'waktunya', 'walau', 'walaupun',
    ↪ 'wong', 'yaitu', 'yakini', 'yakni', 'yang'
]

```

2.3.2 Proses Data Cleaning

```

[11]: # Fungsi untuk membersihkan teks
def cleaning(text):
    text = str(text).lower()                # Buat semua teks
    ↪ menjadi lowercase
    text = re.sub(r'http\S+|www\S+|https\S+', '', text) # Hapus semua link
    text = re.sub(r'\@w+|\#', '', text)      # Hapus mention/tag
    ↪ dan hash dalam hashtag
    text = re.sub(r'\d+', '', text)          # Hapus angka
    text = re.sub(r'[~a-z\s]', ' ', text)    # Hapus karakter
    ↪ selain huruf
    words = text.strip().split()             # Bersihkan whitespace
    ↪ dan pisahkan teks menjadi kata-kata

    # Filter stopword pada dataset
    filtered_words = [
        word for word in words if word not in stopwords_list
    ]

    # Return data bersih
    return " ".join(filtered_words)

# Simpan data bersih di kolom baru
df['cleaned_text'] = df['Tweet'].apply(cleaning)

# Tampilkan contoh dataset (sebagai perbandingan)

```

```
df.head()
```

```
[11]:
```

	Tweet	sentiment	\
0	Ketahui informasi pembagian #PPKM di wilayah J...	Positif	
1	Tempat Ibadah di Wilayah PPKM Level 1 Boleh Be...	Positif	
2	Juru bicara Satgas Covid-19, Wiku Adisasmito m...	Positif	
3	Ketahui informasi pembagian #PPKM di wilayah J...	Positif	
4	Kementerian Agama menerbitkan Surat Edaran Nom...	Positif	

	cleaned_text
0	ketahui informasi pembagian ppkm wilayah jabar...
1	ibadah wilayah ppkm level berkapasitas persen ...
2	juru bicara satgas covid wiku adisasmito menje...
3	ketahui informasi pembagian ppkm wilayah jabar...
4	kementerian agama menerbitkan surat edaran nom...

2.4 Visualisasi Distribusi Kata pada Clean Data

```
[12]: # Gabungkan semua teks mentah menjadi satu string besar
raw_text = " ".join(df['cleaned_text'].astype(str))

# Gunakan wordcloud untuk visualisasi distribusi kata
wordcloud = WordCloud(width=800, height=400, background_color='white').
    generate(raw_text)

# Plot wordcloud
plt.figure(figsize=(12, 8))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud dari Data Bersih')
plt.show()
```


3.2 Data Label Encoding

```
[19]: labels = df['sentiment'].apply(lambda x: 1 if x == 'Positif' else 0).values
labels = labels.reshape(-1, 1)

# Cek distribusi label
unique, counts = np.unique(labels, return_counts=True)

# Jika data tidak seimbang, lakukan oversampling pada kelas minoritas
ros = RandomOverSampler(random_state=42)
x_bal, y_bal = ros.fit_resample(padded_sequences, labels)
```

3.3 Pembagian Dataset

```
[25]: # Bagi data menjadi data train (80%) dan data sisa (20%)
X_train, X_rem, y_train, y_rem = train_test_split(
    x_bal,
    y_bal,
    train_size=0.8,
    random_state=42,
    stratify=y_bal
)

# Bagi data sisa (20%) menjadi 2 bagian untuk test (10%) dan val (10%)
X_val, X_test, y_val, y_test = train_test_split(
    X_rem,
    y_rem,
    test_size=0.5,
    random_state=42,
    stratify=y_rem
)

# Hitung weight
class_weights = compute_class_weight(
    'balanced',
    classes=np.unique(y_train.flatten()),
    y=y_train.flatten()
)

# Ubah weight menjadi dictionary
class_weights_dict = dict(enumerate(class_weights))

# --- Verifikasi Hasil ---
print("Pra-pemrosesan Lanjutan Selesai.")
print(f"Bentuk data training: X={X_train.shape}, y={y_train.shape}")
print(f"Bentuk data validasi: X={X_val.shape}, y={y_val.shape}")
print(f"Bentuk data tes      : X={X_test.shape}, y={y_test.shape}")
```



```
print(f"OOV ratio          : {oov_tokens / total_tokens:.2%}")
```

Pra-pemrosesan Lanjutan Selesai.

Bentuk data training: X=(28329, 14), y=(28329,)

Bentuk data validasi: X=(3541, 14), y=(3541,)

Bentuk data tes : X=(3542, 14), y=(3542,)

OOV ratio : 4.15%

4 Pembagunan Model

4.1 Buat model dengan arsitektur Transformer

```
[30]: # --- A. Positional Encoding (Tidak berubah) ---
def get_positional_encoding(seq_len, d_model):
    positions = np.arange(seq_len)[:, np.newaxis]
    div_term = np.exp(np.arange(0, d_model, 2) * -(np.log(10000.0) / d_model))
    pe = np.zeros((seq_len, d_model))
    pe[:, 0::2] = np.sin(positions * div_term)
    pe[:, 1::2] = np.cos(positions * div_term)
    return tf.cast(pe[np.newaxis, ...], dtype=tf.float32)

# --- B. Fungsi Helper Masking (Hanya Padding Mask) ---
def create_padding_mask_op(seq):
    seq = tf.cast(tf.math.equal(seq, 0), tf.float32)
    return seq[:, tf.newaxis, tf.newaxis, :] # (batch_size, 1, 1, seq_len)

# --- C. Blok Encoder Layer (Tidak berubah) ---
class TransformerEncoderLayer(Layer):
    def __init__(self, d_model, num_heads, dff, rate=0.1):
        super(TransformerEncoderLayer, self).__init__()
        self.mha = MultiHeadAttention(key_dim=d_model, num_heads=num_heads)
        self.ffn = tf.keras.Sequential([
            Dense(dff, activation='relu'),
            Dense(d_model)
        ])
        self.layernorm1 = LayerNormalization(epsilon=1e-6)
        self.layernorm2 = LayerNormalization(epsilon=1e-6)
        self.dropout1 = Dropout(rate)
        self.dropout2 = Dropout(rate)

    def call(self, x, training, mask):
        attn_output = self.mha(query=x, value=x, key=x, attention_mask=mask)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(x + attn_output) # Add & Norm

        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
```

```

        out2 = self.layernorm2(out1 + ffn_output) # Add & Norm
        return out2

# --- Konfigurasi Hyperparameter (Sama) ---
num_layers = 2
d_model = 128
dff = 512
num_heads = 4
dropout_rate = 0.1

# --- Input Layer (Hanya 1 input) ---
encoder_inputs = Input(shape=(maxlen,), name='encoder_input')

# --- Masking (Hanya Padding Mask) ---
enc_padding_mask = tf.keras.layers.Lambda(
    create_padding_mask_op, name='enc_padding_mask'
)(encoder_inputs)

# --- Embedding & Positional Encoding (Hanya untuk Encoder) ---
embedding_layer = Embedding(vocab_size, d_model, name='embedding_layer')

pos_encoding_enc = get_positional_encoding(maxlen, d_model)

enc_emb = embedding_layer(encoder_inputs)
enc_emb = tf.keras.layers.Lambda(lambda x: x * tf.math.sqrt(tf.cast(d_model, tf.
    float32)))(enc_emb)
enc_emb = tf.keras.layers.Add()([enc_emb, pos_encoding_enc[:, :maxlen, :]])
enc_emb = Dropout(dropout_rate)(enc_emb)

# --- ENCODER ---
# Tumpukan Encoder (Sama)
x = enc_emb
for i in range(num_layers):
    x = TransformerEncoderLayer(d_model, num_heads, dff, dropout_rate)(
        x, training=True, mask=enc_padding_mask
    )
encoder_output = x

# 1. Gunakan Global Average Pooling untuk merangkum seluruh kalimat
x = GlobalAveragePooling1D(name='pooling_layer')(encoder_output)

# 2. Tambahkan Dropout untuk regularisasi
x = Dropout(dropout_rate)(x)

# 3. Lapisan Dense terakhir untuk klasifikasi
# Gunakan 1 unit dengan 'sigmoid' untuk sentimen biner (0 atau 1)
final_output = Dense(1, activation='sigmoid', name='sentiment_output')(x)

```

```
# Model sekarang hanya menerima 1 input
model = Model(inputs=encoder_inputs, outputs=final_output)

# --- Kompilasi & Summary ---
# Ganti loss menjadi 'binary_crossentropy' untuk klasifikasi biner
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()
```

Model: "functional_4"

Layer (type)	Output Shape	Param #	Connected to
encoder_input (InputLayer)	(None, 14)	0	-
embedding_layer (Embedding)	(None, 14, 128)	2,560,000	encoder_input[0]...
lambda_2 (Lambda)	(None, 14, 128)	0	embedding_layer[...
add_2 (Add)	(1, 14, 128)	0	lambda_2[0][0]
dropout_7 (Dropout)	(1, 14, 128)	0	add_2[0][0]
enc_padding_mask (Lambda)	(None, 1, 1, 14)	0	encoder_input[0]...
transformer_encode... (TransformerEncode...	(1, 14, 128)	396,032	dropout_7[0][0], enc_padding_mask...
transformer_encode... (TransformerEncode...	(1, 14, 128)	396,032	transformer_enco... enc_padding_mask...
pooling_layer (GlobalAveragePool...	(1, 128)	0	transformer_enco...
dropout_14 (Dropout)	(1, 128)	0	pooling_layer[0]...
sentiment_output (Dense)	(1, 1)	129	dropout_14[0][0]

Total params: 3,352,193 (12.79 MB)

Trainable params: 3,352,193 (12.79 MB)

Non-trainable params: 0 (0.00 B)

5 Data Training

```
[33]: # Mendefinisikan callback EarlyStopping
early_stopping = EarlyStopping(
    monitor='val_loss',          # Metrik yang dipantau
    patience=3,                  # Berhenti jika tidak ada perbaikan selama 3
    ↪epoch
    verbose=1,
    mode='min',                  # Kita ingin meminimalkan 'val_loss'
    restore_best_weights=True    # Kembalikan bobot dari epoch terbaik
)

# Melatih model
history = model.fit(
    X_train,
    y_train,
    batch_size=64,
    epochs=15,
    validation_split=0.2,
    verbose=1,
    callbacks=[early_stopping]
)

print("\nTraining model Transformer selesai.")
```

Epoch 1/15

355/355 24s 32ms/step -

accuracy: 0.7026 - loss: 0.6164 - val_accuracy: 0.9230 - val_loss: 0.2110

Epoch 2/15

355/355 3s 8ms/step -

accuracy: 0.9466 - loss: 0.1448 - val_accuracy: 0.9435 - val_loss: 0.1691

Epoch 3/15

355/355 3s 8ms/step -

accuracy: 0.9673 - loss: 0.0942 - val_accuracy: 0.9474 - val_loss: 0.1700

Epoch 4/15

355/355 3s 8ms/step -

accuracy: 0.9742 - loss: 0.0744 - val_accuracy: 0.9322 - val_loss: 0.2435

Epoch 5/15
355/355 3s 9ms/step -
accuracy: 0.9779 - loss: 0.0650 - val_accuracy: 0.9545 - val_loss: 0.1819
Epoch 5: early stopping
Restoring model weights from the end of the best epoch: 2.

Training model Transformer selesai.

6 Visualisasi Hasil

6.1 Loss vs Accuracy

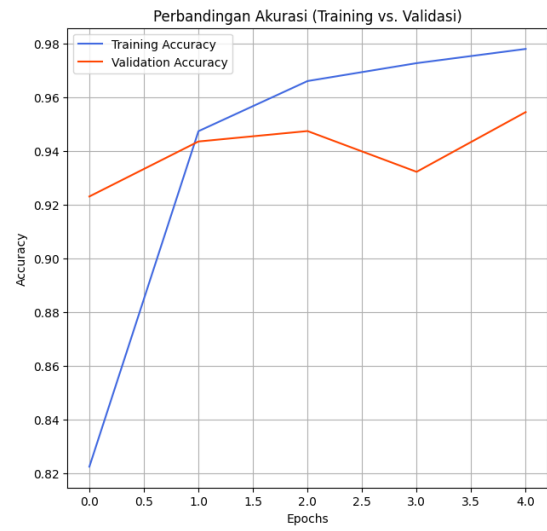
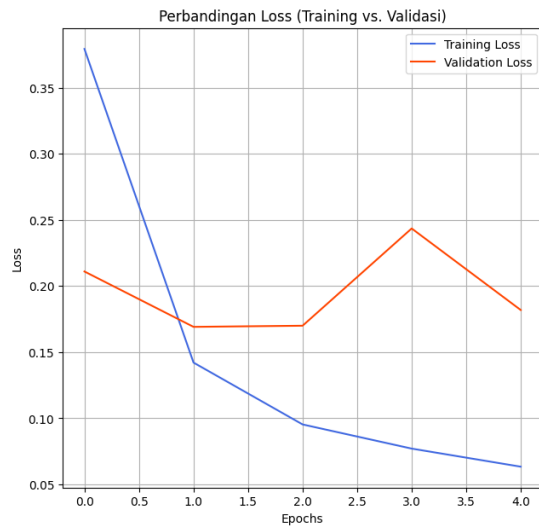
```
[34]: # Mengubah history menjadi DataFrame Pandas untuk kemudahan plotting
history_df = pd.DataFrame(history.history)

# Membuat canvas dengan 2 subplot berdampingan
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))

# --- Plot 1: Loss ---
ax1.set_title('Perbandingan Loss (Training vs. Validasi)')
ax1.plot(history_df['loss'], label='Training Loss', color='royalblue')
ax1.plot(history_df['val_loss'], label='Validation Loss', color='orangered')
ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
ax1.legend()
ax1.grid(True)

# --- Plot 2: Akurasi ---
ax2.set_title('Perbandingan Akurasi (Training vs. Validasi)')
ax2.plot(history_df['accuracy'], label='Training Accuracy', color='royalblue')
ax2.plot(history_df['val_accuracy'], label='Validation Accuracy',
        color='orangered')
ax2.set_xlabel('Epochs')
ax2.set_ylabel('Accuracy')
ax2.legend()
ax2.grid(True)

plt.show()
```



1-of-transformer-summarization-6

November 7, 2025

1 Peringkasan Berita Bahasa Indonesia dengan Transformer

1.1 Pendahuluan

Proyek ini bertujuan untuk membangun model peringkasan teks (Text Summarization) menggunakan arsitektur **Transformer**. Ini adalah model ketiga dalam seri perbandingan kami, setelah sebelumnya mengimplementasikan SimpleRNN dan LSTM. Tujuannya adalah untuk menguji dan membandingkan performa arsitektur berbasis *attention* murni, seperti yang diperkenalkan di paper “Attention Is All You Need”, dengan model berbasis *recurrent* dalam menangani data sekuens panjang untuk peringkasan berita Bahasa Indonesia.

URL Dataset : <https://github.com/kata-ai/indosum>

1.2 Setup & Impor Library

Pada bagian ini, kami mengimpor semua library yang akan dibutuhkan untuk keseluruhan proyek, mulai dari pemrosesan data hingga pembangunan dan evaluasi model.

```
[55]: # Library dasar untuk manipulasi data dan komputasi
import os
import pandas as pd
import numpy as np
import re
import json

# Library untuk visualisasi
import matplotlib.pyplot as plt
from IPython.display import display

# Library TensorFlow dan Keras untuk pra-pemrosesan dan model
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import (
    Input, Embedding, LSTM, Dense, Layer,
    MultiHeadAttention, LayerNormalization, Dropout, Add
)
from tensorflow.keras.models import Model
```

```
print("Semua library berhasil diimpor.")
print("Versi TensorFlow:", tf.__version__)
```

Semua library berhasil diimpor.

Versi TensorFlow: 2.19.0

1.3 Persiapan Dataset (IndoSum)

Pada bagian ini, kami akan menyiapkan dataset IndoSum. Proses ini mencakup pengunduhan, pemuatan file data training, hingga pra-pemrosesan agar data siap digunakan untuk melatih model.

1.3.1 Mengunduh dan Mengekstrak Dataset

Kami akan mengunduh file `indosum.tar.gz` dari Google Drive menggunakan `gdown`, kemudian mengekstraknya di lingkungan Colab menggunakan perintah `tar`.

```
[56]: # Mengunduh dataset indosum.tar.gz dari Google Drive
!gdown --id 10gYbPfXFAv3TbwP1Qcwt_CC9cVWSJaco

# Mengekstrak file .tar.gz
!tar -xzf indosum.tar.gz

print("\nDataset IndoSum berhasil diunduh dan diekstrak.")
```

```
/usr/local/lib/python3.12/dist-packages/gdown/__main__.py:140: FutureWarning:
Option `--id` was deprecated in version 4.3.1 and will be removed in 5.0. You
don't need to pass it anymore to use a file ID.
```

```
warnings.warn(
Downloading...
From (original):
https://drive.google.com/uc?id=10gYbPfXFAv3TbwP1Qcwt_CC9cVWSJaco
From (redirected): https://drive.google.com/uc?id=10gYbPfXFAv3TbwP1Qcwt_CC9cVWSJaco&confirm=t&uuid=b40ddc9c-f9ef-4408-9fc0-06e1a1354308
To: /content/indosum.tar.gz
100% 96.0M/96.0M [00:02<00:00, 36.4MB/s]
indosum/
indosum/test.02.jsonl
indosum/README.txt
indosum/train.05.jsonl
indosum/train.04.jsonl
indosum/test.04.jsonl
indosum/train.01.jsonl
indosum/test.05.jsonl
indosum/test.03.jsonl
indosum/test.01.jsonl
indosum/dev.04.jsonl
indosum/dev.01.jsonl
indosum/dev.02.jsonl
```



```
indosum/train.02.jsonl
indosum/dev.03.jsonl
indosum/dev.05.jsonl
indosum/CHANGELOG.txt
indosum/train.03.jsonl
```

Dataset IndoSum berhasil diunduh dan diekstrak.

1.3.2 Pemuatan Data

Setelah dataset berhasil diunduh dan diekstrak, sekarang kami akan memuat salah satu file data, yaitu train.01.jsonl, ke dalam tabel (DataFrame Pandas).

```
[57]: # --- Pemuatan Data (Gabungkan SEMUA Folds) ---
import pandas as pd

# Buat daftar nama file
train_files = [f'indosum/train.0{i}.jsonl' for i in range(1, 6)]
dev_files = [f'indosum/dev.0{i}.jsonl' for i in range(1, 6)]

# Muat dan gabungkan semua file training
df_train_list = []
for f in train_files:
    df_train_list.append(pd.read_json(f, lines=True))
df_train = pd.concat(df_train_list, ignore_index=True)

# Muat dan gabungkan semua file validasi
df_val_list = []
for f in dev_files:
    df_val_list.append(pd.read_json(f, lines=True))
df_val = pd.concat(df_val_list, ignore_index=True)

print(f"Total data TRAINING (gabungan): {len(df_train)}")
print(f"Total data VALIDASI (gabungan): {len(df_val)}")

# Tampilkan contoh
print("\nContoh 5 Baris Pertama Data Training (Gabungan):")
display(df_train[['paragraphs', 'summary']].head())
```

Total data TRAINING (gabungan): 71353

Total data VALIDASI (gabungan): 3743

Contoh 5 Baris Pertama Data Training (Gabungan):

```
                                paragraphs \
0  [[[Jakarta, , CNN, Indonesia, -, -, Dokter, R...
1  [[[Selfie, ialah, salah, satu, tema, terpanas,...
2  [[[Jakarta, , CNN, Indonesia, -, -, Dinas, Pa...
3  [[[Merdeka.com, -, Indonesia, Corruption, Watc...
```

```

4  [[Merdeka.com, -, Presiden, Joko, Widodo, (, ...

summary
0  [[Dokter, Lula, Kamal, yang, merupakan, selebr...
1  [[Asus, memperkenalkan, , ZenFone, generasi, ...
2  [[Dinas, Pariwisata, Provinsi, Bengkulu, kempa...
3  [[Indonesia, Corruption, Watch, (, ICW, ), mem...
4  [[Jokowi, memimpin, upacara, penurunan, bender...

```

1.3.3 Pra-pemrosesan Teks

Pada tahap ini, kami akan membersihkan teks di kolom paragraphs dan summary. Kami menggunakan fungsi `clean_text` yang dirancang untuk menangani struktur data kompleks dari dataset IndoSum (termasuk list bersarang) dengan mengubahnya menjadi satu string tunggal. Setelah itu, kami akan menambahkan token `<start>` dan `<end>` pada setiap kalimat ringkasan yang sudah bersih untuk mempersiapkannya sebagai input bagi decoder.

```

[58]: # Fungsi clean_text
def clean_text(data):
    text = str(data).lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    text = re.sub(r'\@w+|\#', '', text)
    text = re.sub(r'\"\"[\],\']', '', text)
    text = re.sub(r'^a-z0-9\s\.-', '', text)
    text = re.sub(r'(?!\d)\-|-(?!d)', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    text = re.sub(r'\s\.', '.', text)
    text = text.replace(u'xa0', u' ')
    return text

# Menerapkan fungsi cleaning ke df_train
print("Membersihkan data training...")
df_train['artikel_bersih'] = df_train['paragraphs'].apply(clean_text)
df_train['ringkasan_bersih'] = df_train['summary'].apply(clean_text)
df_train['ringkasan_decoder'] = df_train['ringkasan_bersih'].apply(lambda x:
    ↪ '<start> ' + x + ' <end>')

# Menerapkan fungsi cleaning ke df_val
print("Membersihkan data validasi...")
df_val['artikel_bersih'] = df_val['paragraphs'].apply(clean_text)
df_val['ringkasan_bersih'] = df_val['summary'].apply(clean_text)
df_val['ringkasan_decoder'] = df_val['ringkasan_bersih'].apply(lambda x:
    ↪ '<start> ' + x + ' <end>')

print("Pembersihan selesai.")

```

Membersihkan data training...

Membersihkan data validasi...

Pembersihan selesai.

```
[59]: print("--- Memfilter Ringkasan Kosong ---")

# --- Cek Training ---
len_before_train = len(df_train)
print(f"Jumlah data training SEBELUM filter: {len_before_train}")

# Filter: Hapus baris di mana 'ringkasan_bersih' adalah string kosong ""
df_train = df_train[df_train['ringkasan_bersih'].str.strip() != ''].
    ↪reset_index(drop=True)

len_after_train = len(df_train)
print(f"Jumlah data training SETELAH filter: {len_after_train}")
print(f"Dibuang {len_before_train - len_after_train} sampel ringkasan kosong.")
print("-" * 20)

# --- Cek Validasi ---
len_before_val = len(df_val)
print(f"Jumlah data validasi SEBELUM filter: {len_before_val}")

df_val = df_val[df_val['ringkasan_bersih'].str.strip() != ''].
    ↪reset_index(drop=True)

len_after_val = len(df_val)
print(f"Jumlah data validasi SETELAH filter: {len_after_val}")
print(f"Dibuang {len_before_val - len_after_val} sampel ringkasan kosong.")
print("-----")
```

```
--- Memfilter Ringkasan Kosong ---
Jumlah data training SEBELUM filter: 71353
Jumlah data training SETELAH filter: 71353
Dibuang 0 sampel ringkasan kosong.
-----
Jumlah data validasi SEBELUM filter: 3743
Jumlah data validasi SETELAH filter: 3743
Dibuang 0 sampel ringkasan kosong.
-----
```

```
[60]: print("--- Pengecekan Duplikat ---")

# === Cek Data Training (df_train) ===
len_before_train = len(df_train)
print(f"Jumlah data training SEBELUM dicek: {len_before_train}")

# Hitung duplikat berdasarkan artikel dan ringkasan yang sudah bersih
# Kita anggap duplikat jika artikel DAN ringkasannya sama persis
```

```

duplicates_train = df_train.duplicated(subset=['artikel_bersih',
↳ 'ringkasan_bersih'])
num_duplicates_train = duplicates_train.sum()
print(f"Ditemukan {num_duplicates_train} duplikat di data training.")

# Hapus duplikat jika ada
if num_duplicates_train > 0:
    # Kita gunakan '~' (negasi) untuk mengambil semua baris yang BUKAN duplikat
    df_train = df_train[~duplicates_train].reset_index(drop=True)
    print(f"Duplikat di data training telah dihapus.")

len_after_train = len(df_train)
print(f"Jumlah data training SETELAH dicek: {len_after_train}")
print("-" * 20)

# === Cek Data Validasi (df_val) ===
len_before_val = len(df_val)
print(f"Jumlah data validasi SEBELUM dicek: {len_before_val}")

duplicates_val = df_val.duplicated(subset=['artikel_bersih',
↳ 'ringkasan_bersih'])
num_duplicates_val = duplicates_val.sum()
print(f"Ditemukan {num_duplicates_val} duplikat di data validasi.")

if num_duplicates_val > 0:
    df_val = df_val[~duplicates_val].reset_index(drop=True)
    print(f"Duplikat di data validasi telah dihapus.")

len_after_val = len(df_val)
print(f"Jumlah data validasi SETELAH dicek: {len_after_val}")
print("-----")

```

```

--- Pengecekan Duplikat ---
Jumlah data training SEBELUM dicek: 71353
Ditemukan 51993 duplikat di data training.
Duplikat di data training telah dihapus.
Jumlah data training SETELAH dicek: 19360
-----

Jumlah data validasi SEBELUM dicek: 3743
Ditemukan 266 duplikat di data validasi.
Duplikat di data validasi telah dihapus.
Jumlah data validasi SETELAH dicek: 3477
-----

```

```
[61]: print("--- Pengecekan Kebocoran Data (Train vs. Val) ---")
```

```
# Kolom yang akan kita gunakan sebagai kunci unik
```

```

key_columns = ['artikel_bersih', 'ringkasan_bersih']

# Lakukan "inner merge" untuk menemukan baris yang identik di kedua set
# Ini akan membuat DataFrame baru yang HANYA berisi data yang bocor
leaked_data = pd.merge(
    df_train[key_columns],
    df_val[key_columns],
    on=key_columns,
    how='inner'
)

# Laporkan hasilnya
num_leaked = len(leaked_data)

if num_leaked > 0:
    print(f" PERINGATAN: Ditemukan {num_leaked} sampel yang identik (bocor) antara data training dan data validasi!")
    print("Ini adalah penyebab utama val_loss Anda terlihat 'terlalu bagus'.")
    print("\nContoh 5 data yang bocor:")
    display(leaked_data.head())
else:
    print(" BAGUS: Tidak ditemukan kebocoran data (sampel identik) antara training dan validasi.")

print("-----")

```

--- Pengecekan Kebocoran Data (Train vs. Val) ---

PERINGATAN: Ditemukan 3342 sampel yang identik (bocor) antara data training dan data validasi!

Ini adalah penyebab utama val_loss Anda terlihat 'terlalu bagus'.

Contoh 5 data yang bocor:

	artikel_bersih \
0	jakarta cnn indonesia pebalap yamaha valentino...
1	pemerintah per tanggal 31 oktober 2017 akan me...
2	jakarta antara news sebuah hotel di belgia men...
3	jakarta cnn indonesia liam payne kini mulai be...
4	harry kane melanjutkan performa impresifnya di...

	ringkasan_bersih
0	pebalap yamaha valentino rossi punya pandangan...
1	untuk memudahkan pengguna jalan tol dalam meng...
2	sebuah hotel di belgia menawarkan sewa ikan un...
3	liam payne kini mulai bergerak menyusul teman ...
4	harry kane melanjutkan performa impresifnya di...

```
[62]: # Jalankan ini HANYA JIKA num_leaked > 0 dari sel sebelumnya

print(f"\n--- Membersihkan {num_leaked} Data Bocor dari Set Validasi ---")

# Lakukan 'left merge' antara df_val dan leaked_data dengan penanda
df_val_merged = df_val.merge(
    leaked_data,
    on=key_columns,
    how='left',
    indicator=True # Ini akan menambah kolom '_merge'
)

# Kolom '_merge' akan berisi 'both' jika ada di leaked_data,
# dan 'left_only' jika tidak.

# Kita ambil HANYA baris yang 'left_only'
df_val_cleaned = df_val_merged[df_val_merged['_merge'] == 'left_only'].
    drop(columns=['_merge'])

print(f"Jumlah data validasi SEBELUM dibersihkan: {len(df_val)}")
print(f"Jumlah data validasi SETELAH dibersihkan: {len(df_val_cleaned)}")

# Update DataFrame validasi Anda
df_val = df_val_cleaned

print("Set validasi (df_val) telah bersih dari kebocoran.")
```

```
--- Membersihkan 3342 Data Bocor dari Set Validasi ---
Jumlah data validasi SEBELUM dibersihkan: 3477
Jumlah data validasi SETELAH dibersihkan: 135
Set validasi (df_val) telah bersih dari kebocoran.
```

```
[63]: print("--- Menggabungkan Sisa Validasi ke Training ---")
print(f"Data training sebelum digabung: {len(df_train)}")
print(f"Data validasi sisa: {len(df_val)}")

# Gabungkan df_val (yang 135) ke df_train
df_train = pd.concat([df_train, df_val], ignore_index=True)

print(f"Data training SETELAH digabung: {len(df_train)}")

# Kosongkan df_val agar tidak terpakai lagi
df_val = pd.DataFrame(columns=df_train.columns)
print("Set validasi (df_val) telah dikosongkan.")
```

```
--- Menggabungkan Sisa Validasi ke Training ---
```

Data training sebelum digabung: 19360
Data validasi sisa: 135
Data training SETELAH digabung: 19495
Set validasi (df_val) telah dikosongkan.

1.3.4 Tokenisasi & Persiapan Data Training

Pada tahap ini, kami akan melakukan beberapa langkah kunci pada data dari file train.01.jsonl yang telah dimuat ke dalam DataFrame df: - Tokenisasi: Kami membuat kamus kata (vocabulary) dan mengubah setiap kalimat menjadi sekuens ID angka. - Padding: Semua sekuens diseragamkan panjangnya. - Persiapan Target Decoder: Kami membuat “kunci jawaban” untuk decoder, yaitu sekuens yang digeser satu langkah ke depan.

```
[64]: # --- 1. TOKENISASI & PADDING ---
vocab_size = 10000
oov_token = '<OOV>'

# Buat kamus HANYA dari data training (yang sudah digabung)
all_texts_train_only = (
    df_train['artikel_bersih'].tolist() +
    df_train['ringkasan_decoder'].tolist()
)

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>', filters='')
tokenizer.fit_on_texts(all_texts_train_only)
print("Tokenizer berhasil di-fit HANYA pada data training (gabungan).")

# Menampilkan jumlah data final setelah filtering
print(f"Total data yang akan ditokenisasi (setelah filtering): {len(df_train)}")

# --- 2. Tentukan Max Length (Gunakan data training) ---
train_input_seq = tokenizer.texts_to_sequences(df_train['artikel_bersih'])
train_target_seq = tokenizer.texts_to_sequences(df_train['ringkasan_decoder'])

# Tentukan max_length secara dinamis HANYA dari data training
max_len_input_asli = max(len(s) for s in train_input_seq)
max_len_target = max(len(s) for s in train_target_seq)

# Batasi max_len_input untuk efisiensi
max_len_input = 850

print(f"Panjang maksimal artikel (ASLI): {max_len_input_asli}")
print(f"Panjang maksimal artikel (DIPOTONG): {max_len_input}")
print(f"Panjang maksimal ringkasan (dari train gabungan): {max_len_target}")

# --- 3. Proses Data Training ---
```

```

# truncating='post' akan memotong akhir artikel jika > 850
encoder_input_data = pad_sequences(train_input_seq, maxlen=max_len_input,
    ↳padding='post', truncating='post')
decoder_input_data = pad_sequences(train_target_seq, maxlen=max_len_target,
    ↳padding='post', truncating='post')

decoder_target_data = np.zeros_like(decoder_input_data, dtype=np.int32)
for i, seq in enumerate(decoder_input_data):
    decoder_target_data[i, :-1] = seq[1:]

# Print ini akan mengonfirmasi jumlah data setelah di-padding
print(f"\nBentuk data input encoder (TRAIN): {encoder_input_data.shape}")
print(f"Bentuk data input decoder (TRAIN): {decoder_input_data.shape}")
print(f"Bentuk data target decoder (TRAIN): {decoder_target_data.shape}")

```

Tokenizer berhasil di-fit HANYA pada data training (gabungan).
 Total data yang akan ditokenisasi (setelah filtering): 19495
 Panjang maksimal artikel (ASLI): 1262
 Panjang maksimal artikel (DIPOTONG): 850
 Panjang maksimal ringkasan (dari train gabungan): 88

Bentuk data input encoder (TRAIN): (19495, 850)
 Bentuk data input decoder (TRAIN): (19495, 88)
 Bentuk data target decoder (TRAIN): (19495, 88)

```

[65]: from sklearn.model_selection import train_test_split

print("--- Membuat Validation Split yang Baru (90/10) ---")

# Data kita ada 3: encoder_input, decoder_input, decoder_target
# Kita split ketiganya secara bersamaan agar tetap sinkron
enc_in_train, enc_in_val, \
dec_in_train, dec_in_val, \
dec_target_train, dec_target_val = train_test_split(
    encoder_input_data,      # X1
    decoder_input_data,      # X2
    decoder_target_data,     # y
    test_size=0.1,           # 10% untuk validasi (sekitar 1950 sampel)
    random_state=42          # Agar hasilnya konsisten
)

print(f"Total data: {len(encoder_input_data)}")
print(f>Data Training Baru: {len(enc_in_train)}")
print(f>Data Validasi Baru: {len(enc_in_val)}")

# Siapkan format data untuk model.fit()
# Data training (90%)

```



```

training_set = (
    [enc_in_train, dec_in_train],
    dec_target_train
)

# Data validasi (10%)
validation_set = (
    [enc_in_val, dec_in_val],
    dec_target_val
)

print("\nSplit data training (90%) dan validasi (10%) baru telah siap.")

```

--- Membuat Validation Split yang Baru (90/10) ---

Total data: 19495

Data Training Baru: 17545

Data Validasi Baru: 1950

Split data training (90%) dan validasi (10%) baru telah siap.

1.4 Arsitektur & Training Model (Prototyping)

Pada bagian ini, kami akan membangun arsitektur model sequence-to-sequence menggunakan arsitektur **Transformer**. Berbeda dari model RNN/LSTM, model ini tidak menggunakan *recurrence*, melainkan murni berbasis *self-attention* untuk menangani dependensi jangka panjang.

1.4.1 Arsitektur Model

Arsitektur ini terdiri dari dua komponen utama: 1. **Encoder**: Membaca sekuens artikel dan menghasilkan representasi kontekstual. 2. **Decoder**: Menggunakan representasi dari encoder dan output sebelumnya untuk menghasilkan sekuens ringkasan kata per kata.

Kami akan membangun komponen-komponen Transformer (Positional Encoding, Encoder Layer, Decoder Layer) terlebih dahulu sebelum merakit model final.

```

[66]: # --- 1. KOMPONEN-KOMPONEN TRANSFORMER (VERSI DIPERBAIKI) ---
from tensorflow.keras.layers import Lambda, Add
from tensorflow.keras.callbacks import EarlyStopping

# --- A. Positional Encoding ---
def get_positional_encoding(seq_len, d_model):
    positions = np.arange(seq_len)[:, np.newaxis]
    div_term = np.exp(np.arange(0, d_model, 2) * -(np.log(10000.0) / d_model))

    pe = np.zeros((seq_len, d_model))
    pe[:, 0::2] = np.sin(positions * div_term)
    pe[:, 1::2] = np.cos(positions * div_term)

```

```

    return tf.cast(pe[np.newaxis, ...], dtype=tf.float32)

# --- B. Fungsi Helper Masking (untuk di dalam Lambda) ---
def create_padding_mask_op(seq):
    seq = tf.cast(tf.math.equal(seq, 0), tf.float32)
    return seq[:, tf.newaxis, tf.newaxis, :] # (batch_size, 1, 1, seq_len)

def create_look_ahead_mask_op(seq): # Menggabungkan look-ahead dan padding
    size = tf.shape(seq)[1]
    look_ahead_mask = 1 - tf.linalg.band_part(tf.ones((size, size)), -1, 0)
    padding_mask = create_padding_mask_op(seq)
    combined_mask = tf.maximum(padding_mask, look_ahead_mask)
    return combined_mask

# --- C. Blok Encoder Layer ---
class TransformerEncoderLayer(Layer):
    def __init__(self, d_model, num_heads, dff, rate=0.1):
        super(TransformerEncoderLayer, self).__init__()
        self.mha = MultiHeadAttention(key_dim=d_model, num_heads=num_heads)
        self.ffn = tf.keras.Sequential([
            Dense(dff, activation='relu'),
            Dense(d_model)
        ])
        self.layernorm1 = LayerNormalization(epsilon=1e-6)
        self.layernorm2 = LayerNormalization(epsilon=1e-6)
        self.dropout1 = Dropout(rate)
        self.dropout2 = Dropout(rate)

    def call(self, x, training=None, mask=None):
        attn_output = self.mha(query=x, value=x, key=x, attention_mask=mask)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(x + attn_output) # Add & Norm

        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
        out2 = self.layernorm2(out1 + ffn_output) # Add & Norm
        return out2

# --- D. Blok Decoder Layer ---
class TransformerDecoderLayer(Layer):
    def __init__(self, d_model, num_heads, dff, rate=0.1):
        super(TransformerDecoderLayer, self).__init__()
        self.mha1 = MultiHeadAttention(key_dim=d_model, num_heads=num_heads) #  

        ↪Masked
        self.mha2 = MultiHeadAttention(key_dim=d_model, num_heads=num_heads) #  

        ↪Encoder-Decoder

```

```

        self.ffn = tf.keras.Sequential([
            Dense(dff, activation='relu'),
            Dense(d_model)
        ])
        self.layernorm1 = LayerNormalization(epsilon=1e-6)
        self.layernorm2 = LayerNormalization(epsilon=1e-6)
        self.layernorm3 = LayerNormalization(epsilon=1e-6)
        self.dropout1 = Dropout(rate)
        self.dropout2 = Dropout(rate)
        self.dropout3 = Dropout(rate)

    def call(self, x, enc_output, training=None, look_ahead_mask=None,
padding_mask=None):
        attn1 = self.mha1(query=x, value=x, key=x,
attention_mask=look_ahead_mask)
        attn1 = self.dropout1(attn1, training=training)
        out1 = self.layernorm1(x + attn1) # Add & Norm

        attn2 = self.mha2(query=out1, value=enc_output, key=enc_output,
attention_mask=padding_mask)
        attn2 = self.dropout2(attn2, training=training)
        out2 = self.layernorm2(out1 + attn2) # Perbaikan: out1 + attn2

        ffn_output = self.ffn(out2)
        ffn_output = self.dropout3(ffn_output, training=training)
        out3 = self.layernorm3(out2 + ffn_output) # Add & Norm
        return out3

# --- 2. MERAKIT MODEL TRANSFORMER ---

# --- Konfigurasi Hyperparameter ---
num_layers = 1
d_model = 64
dff = 512
num_heads = 4
dropout_rate = 0.5
# --- -----

# --- Input Layer ---
encoder_inputs = Input(shape=(max_len_input,), name='encoder_input')
decoder_inputs = Input(shape=(max_len_target,), name='decoder_input')

# --- Masking (Menggunakan Lambda Layers) ---
enc_padding_mask = Lambda(
    create_padding_mask_op, name='enc_padding_mask'
)(encoder_inputs)

```

```

combined_mask = Lambda(
    create_look_ahead_mask_op, name='combined_mask'
)(decoder_inputs)

# --- Embedding & Positional Encoding (Menggunakan Keras Layers) ---
embedding_layer = Embedding(vocab_size, d_model, name='embedding_layer')
pos_encoding_enc = get_positional_encoding(max_len_input, d_model)
pos_encoding_dec = get_positional_encoding(max_len_target, d_model)

enc_emb = embedding_layer(encoder_inputs)
enc_emb = Lambda(lambda x: x * tf.math.sqrt(tf.cast(d_model, tf.
    float32)))(enc_emb)
enc_emb = Add()(enc_emb, pos_encoding_enc[:, :max_len_input, :])
enc_emb = Dropout(dropout_rate)(enc_emb)

dec_emb = embedding_layer(decoder_inputs)
dec_emb = Lambda(lambda x: x * tf.math.sqrt(tf.cast(d_model, tf.
    float32)))(dec_emb)
dec_emb = Add()(dec_emb, pos_encoding_dec[:, :max_len_target, :])
dec_emb = Dropout(dropout_rate)(dec_emb)

# --- ENCODER ---
x = enc_emb
for i in range(num_layers):
    x = TransformerEncoderLayer(d_model, num_heads, dff, dropout_rate)(
        x, mask=enc_padding_mask
    )
encoder_output = x

# --- DECODER ---
y = dec_emb
for i in range(num_layers):
    y = TransformerDecoderLayer(d_model, num_heads, dff, dropout_rate)(
        y, encoder_output,
        look_ahead_mask=combined_mask,
        padding_mask=enc_padding_mask
    )
decoder_output = y

# --- Final Output Layer ---
decoder_dense = Dense(vocab_size, activation='softmax', name='decoder_dense')
final_output = decoder_dense(decoder_output)

# --- Gabungkan Menjadi Model Training ---
model = Model(inputs=[encoder_inputs, decoder_inputs], outputs=final_output)

# --- Kompilasi & Summary ---

```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
             metrics=['accuracy'])  
model.summary()
```

Model: "functional_29"

Layer (type)	Output Shape	Param #	Connected to
decoder_input (InputLayer)	(None, 88)	0	-
encoder_input (InputLayer)	(None, 850)	0	-
embedding_layer (Embedding)	(None, 88, 64)	640,000	encoder_input[0]... decoder_input[0]...
lambda_14 (Lambda)	(None, 850, 64)	0	embedding_layer[...]
lambda_15 (Lambda)	(None, 88, 64)	0	embedding_layer[...]
add_14 (Add)	(1, 850, 64)	0	lambda_14[0][0]
add_15 (Add)	(1, 88, 64)	0	lambda_15[0][0]
dropout_94 (Dropout)	(1, 850, 64)	0	add_14[0][0]
enc_padding_mask (Lambda)	(None, 1, 1, 850)	0	encoder_input[0]...
dropout_95 (Dropout)	(1, 88, 64)	0	add_15[0][0]
transformer_encode... (TransformerEncode...	(1, 850, 64)	132,736	dropout_94[0][0], enc_padding_mask...
combined_mask (Lambda)	(None, 1, 88, 88)	0	decoder_input[0]...
transformer_decode... (TransformerDecode...	(1, 88, 64)	199,232	dropout_95[0][0], transformer_encode... combined_mask[0]... enc_padding_mask...
decoder_dense	(1, 88, 10000)	650,000	transformer_decode...

(Dense)

Total params: 1,621,968 (6.19 MB)

Trainable params: 1,621,968 (6.19 MB)

Non-trainable params: 0 (0.00 B)

1.4.2 Proses Training

Sekarang kami akan melatih model Transformer dengan data yang sudah disiapkan (data training yang jauh lebih besar). Kami juga akan menambahkan **EarlyStopping** untuk mencegah overfitting dan mendapatkan bobot terbaik secara otomatis.

```
[67]: print("\nMemulai proses training (DENGAN VALIDASI & CALLBACK)...")

# Definisikan callback EarlyStopping
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=3,
    verbose=1,
    mode='min',
    restore_best_weights=True
)

# Melatih model
history = model.fit(
    training_set[0],
    training_set[1],
    batch_size=32,
    epochs=100,
    validation_data=validation_set,
    verbose=1,
    callbacks=[early_stopping]
)

print("\nTraining model Transformer selesai.")
```

Memulai proses training (DENGAN VALIDASI & CALLBACK)...

Epoch 1/100

549/549 69s 96ms/step -

accuracy: 0.3759 - loss: 5.8295 - val_accuracy: 0.4953 - val_loss: 3.6463

Epoch 2/100

549/549 40s 73ms/step -

accuracy: 0.4787 - loss: 3.7819 - val_accuracy: 0.3307 - val_loss: 2.8599
 Epoch 3/100
 549/549 40s 73ms/step -
 accuracy: 0.5395 - loss: 3.1153 - val_accuracy: 0.4298 - val_loss: 2.5820
 Epoch 4/100
 549/549 40s 73ms/step -
 accuracy: 0.5759 - loss: 2.7677 - val_accuracy: 0.7754 - val_loss: 1.2572
 Epoch 5/100
 549/549 40s 73ms/step -
 accuracy: 0.6009 - loss: 2.5481 - val_accuracy: 0.8671 - val_loss: 0.9589
 Epoch 6/100
 549/549 40s 73ms/step -
 accuracy: 0.6184 - loss: 2.3921 - val_accuracy: 0.8965 - val_loss: 0.6040
 Epoch 7/100
 549/549 40s 73ms/step -
 accuracy: 0.6305 - loss: 2.2851 - val_accuracy: 0.9224 - val_loss: 0.4660
 Epoch 8/100
 549/549 40s 72ms/step -
 accuracy: 0.6403 - loss: 2.2094 - val_accuracy: 0.9383 - val_loss: 0.3768
 Epoch 9/100
 549/549 40s 72ms/step -
 accuracy: 0.6482 - loss: 2.1462 - val_accuracy: 0.9502 - val_loss: 0.3026
 Epoch 10/100
 549/549 39s 72ms/step -
 accuracy: 0.6551 - loss: 2.0914 - val_accuracy: 0.9579 - val_loss: 0.2561
 Epoch 11/100
 549/549 40s 72ms/step -
 accuracy: 0.6600 - loss: 2.0469 - val_accuracy: 0.9669 - val_loss: 0.2150
 Epoch 12/100
 549/549 39s 72ms/step -
 accuracy: 0.6642 - loss: 2.0096 - val_accuracy: 0.9709 - val_loss: 0.1845
 Epoch 13/100
 549/549 39s 72ms/step -
 accuracy: 0.6687 - loss: 1.9842 - val_accuracy: 0.9738 - val_loss: 0.1688
 Epoch 14/100
 549/549 39s 72ms/step -
 accuracy: 0.6720 - loss: 1.9583 - val_accuracy: 0.9775 - val_loss: 0.1498
 Epoch 15/100
 549/549 39s 72ms/step -
 accuracy: 0.6756 - loss: 1.9317 - val_accuracy: 0.9799 - val_loss: 0.1338
 Epoch 16/100
 549/549 39s 72ms/step -
 accuracy: 0.6783 - loss: 1.9139 - val_accuracy: 0.9818 - val_loss: 0.1228
 Epoch 17/100
 549/549 39s 71ms/step -
 accuracy: 0.6796 - loss: 1.8992 - val_accuracy: 0.9817 - val_loss: 0.1148
 Epoch 18/100
 549/549 40s 73ms/step -

accuracy: 0.6830 - loss: 1.8756 - val_accuracy: 0.9837 - val_loss: 0.1061
 Epoch 19/100
 549/549 40s 73ms/step -
 accuracy: 0.6854 - loss: 1.8594 - val_accuracy: 0.9854 - val_loss: 0.0969
 Epoch 20/100
 549/549 41s 74ms/step -
 accuracy: 0.6868 - loss: 1.8503 - val_accuracy: 0.9856 - val_loss: 0.0954
 Epoch 21/100
 549/549 41s 74ms/step -
 accuracy: 0.6880 - loss: 1.8413 - val_accuracy: 0.9869 - val_loss: 0.0868
 Epoch 22/100
 549/549 41s 74ms/step -
 accuracy: 0.6898 - loss: 1.8241 - val_accuracy: 0.9868 - val_loss: 0.0840
 Epoch 23/100
 549/549 41s 74ms/step -
 accuracy: 0.6919 - loss: 1.8165 - val_accuracy: 0.9877 - val_loss: 0.0787
 Epoch 24/100
 549/549 41s 74ms/step -
 accuracy: 0.6930 - loss: 1.8050 - val_accuracy: 0.9878 - val_loss: 0.0767
 Epoch 25/100
 549/549 41s 74ms/step -
 accuracy: 0.6949 - loss: 1.7934 - val_accuracy: 0.9872 - val_loss: 0.0746
 Epoch 26/100
 549/549 41s 74ms/step -
 accuracy: 0.6951 - loss: 1.7919 - val_accuracy: 0.9884 - val_loss: 0.0707
 Epoch 27/100
 549/549 41s 74ms/step -
 accuracy: 0.6969 - loss: 1.7748 - val_accuracy: 0.9889 - val_loss: 0.0671
 Epoch 28/100
 549/549 41s 74ms/step -
 accuracy: 0.6972 - loss: 1.7742 - val_accuracy: 0.9890 - val_loss: 0.0671
 Epoch 29/100
 549/549 41s 74ms/step -
 accuracy: 0.6980 - loss: 1.7629 - val_accuracy: 0.9883 - val_loss: 0.0682
 Epoch 30/100
 549/549 40s 74ms/step -
 accuracy: 0.6991 - loss: 1.7546 - val_accuracy: 0.9897 - val_loss: 0.0613
 Epoch 31/100
 549/549 40s 73ms/step -
 accuracy: 0.6996 - loss: 1.7489 - val_accuracy: 0.9905 - val_loss: 0.0564
 Epoch 32/100
 549/549 40s 73ms/step -
 accuracy: 0.7015 - loss: 1.7345 - val_accuracy: 0.9903 - val_loss: 0.0553
 Epoch 33/100
 549/549 41s 74ms/step -
 accuracy: 0.7025 - loss: 1.7216 - val_accuracy: 0.9918 - val_loss: 0.0501
 Epoch 34/100
 549/549 40s 73ms/step -


```

accuracy: 0.7029 - loss: 1.7172 - val_accuracy: 0.9924 - val_loss: 0.0459
Epoch 35/100
549/549          40s 73ms/step -
accuracy: 0.7038 - loss: 1.7084 - val_accuracy: 0.9918 - val_loss: 0.0489
Epoch 36/100
549/549          40s 73ms/step -
accuracy: 0.7054 - loss: 1.6982 - val_accuracy: 0.9908 - val_loss: 0.0500
Epoch 37/100
549/549          40s 73ms/step -
accuracy: 0.7050 - loss: 1.6989 - val_accuracy: 0.9921 - val_loss: 0.0468
Epoch 37: early stopping
Restoring model weights from the end of the best epoch: 34.

```

Training model Transformer selesai.

1.5 Evaluasi dan Uji Coba

1.5.1 Analisis Kurva Pembelajaran

Ini adalah kasus “kegagalan” yang sangat menarik dan membingungkan:

Log Training: Kelihatan sempurna. `val_loss` turun dengan indah dari 3.64 ke 0.0459. EarlyStopping bekerja persis seperti yang diharapkan, berhenti di Epoch 37 dan mengembalikan bobot terbaik dari Epoch 34.

Hasil Uji Coba: Gagal total. Ringkasan Model (Greedy): (Tidak menghasilkan output).

1.5.2 Visualisasi Kurva Pembelajaran

Sama seperti model sebelumnya, kami akan memvisualisasikan metrik `loss` dan `accuracy` untuk melihat bagaimana proses training model berjalan.

```

[74]: import pandas as pd
import matplotlib.pyplot as plt

# Mengubah history menjadi DataFrame Pandas
history_df = pd.DataFrame(history.history)

# Membuat canvas dengan 2 subplot berdampingan
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))

# --- Plot 1: Loss ---
ax1.set_title('Training Loss')
ax1.plot(history_df['loss'], label='Training Loss', color='royalblue')
ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
ax1.legend()
ax1.grid(True)

# --- Plot 2: Akurasi ---

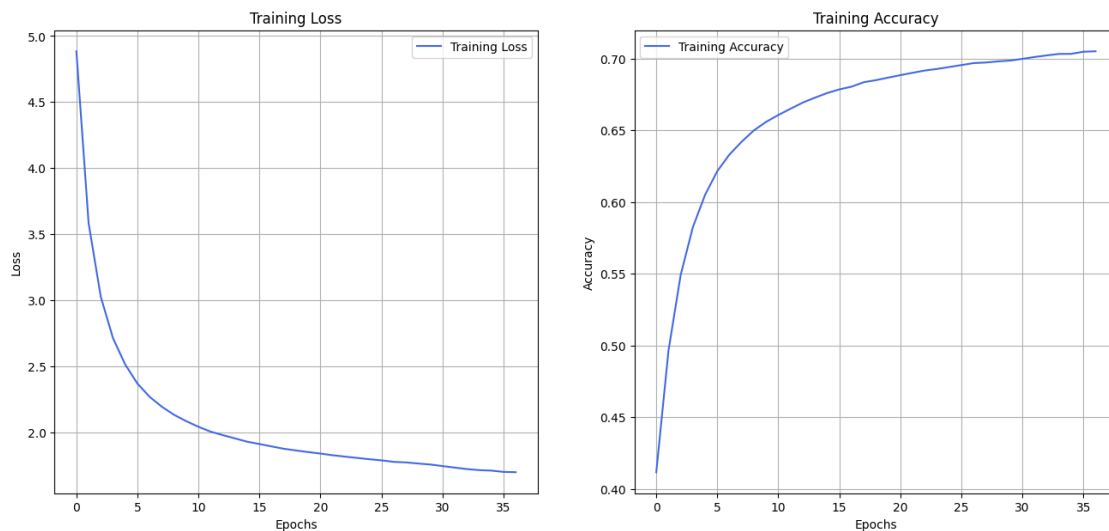
```

```

ax2.set_title('Training Accuracy')
ax2.plot(history_df['accuracy'], label='Training Accuracy', color='royalblue')
ax2.set_xlabel('Epochs')
ax2.set_ylabel('Accuracy')
ax2.legend()
ax2.grid(True)

plt.show()

```



1.5.3 Uji Coba Model (Inferensi)

Proses inferensi pada Transformer sedikit berbeda. Kami tidak perlu membuat model **encoder** dan **decoder** terpisah. Sebaliknya, kami akan memanggil model yang sudah dilatih (**model**) secara berulang dalam sebuah *loop*, mensimulasikan proses *auto-regressive* (kata per kata) secara manual.

Fungsi decode ini juga akan mengadopsi **Temperature Sampling** dari *notebook* LSTM untuk perbandingan yang adil.

```

[69]: # --- 1. FUNGSI HELPER UNTUK MASKING ---
# Kita perlu fungsi-fungsi ini lagi untuk membuat mask secara dinamis
def create_padding_mask(seq):
    seq = tf.cast(tf.math.equal(seq, 0), tf.float32)
    return seq[:, tf.newaxis, tf.newaxis, :] # (batch_size, 1, 1, seq_len)

def create_look_ahead_mask(size):
    mask = 1 - tf.linalg.band_part(tf.ones((size, size)), -1, 0)
    return mask # (seq_len, seq_len)

# --- 2. FUNGSI DECODE UNTUK TRANSFORMER ---

```

```

# Kamus untuk konversi ID ke kata
reverse_word_index = {v: k for k, v in tokenizer.word_index.items()}

def decode_sequence_transformer(input_seq, temperature=1.0):
    # 1. Siapkan Encoder Input (input_seq sudah benar, shape (1, max_len_input))

    # 2. Siapkan Decoder Input awal
    # Mulai dengan token <start>
    decoder_input = np.zeros((1, max_len_target), dtype=np.int32)
    decoder_input[0, 0] = tokenizer.word_index['<start>']

    output_sentence = []

    # 3. Loop Auto-regresif
    for i in range(max_len_target - 1):
        # Prediksi dengan model
        predictions = model.predict([input_seq, decoder_input], verbose=0)

        # Ambil prediksi HANYA untuk token ke-i (token berikutnya)
        predictions_i = predictions[:, i, :]

        # --- Logika Temperature Sampling ---
        scaled_logits = predictions_i / temperature
        sampled_token_index = tf.random.categorical(scaled_logits,
        ↪ num_samples=1)[0, 0].numpy()

        sampled_word = reverse_word_index.get(sampled_token_index, '<unk>')

        if sampled_word == '<end>':
            break

        if sampled_word != '<unk>':
            output_sentence.append(sampled_word)

        # 4. Update input untuk langkah berikutnya
        if i < max_len_target - 1:
            decoder_input[0, i + 1] = sampled_token_index

    return " ".join(output_sentence)

print("Fungsi untuk inferensi Transformer (decode_sequence_transformer) siap_
    ↪ digunakan.")

```

Fungsi untuk inferensi Transformer (decode_sequence_transformer) siap digunakan.

1.5.4 Evaluasi Kuantitatif (ROUGE)

Untuk perbandingan yang objektif, kami akan menggunakan metrik **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)**. Metrik ini adalah standar untuk mengevaluasi peringkat teks dengan membandingkan ringkasan yang dihasilkan model (*candidate*) dengan ringkasan referensi (*ground truth*).

Kami akan menghitung: * **ROUGE-1**: Tumpang tindih *unigram* (kata per kata). * **ROUGE-2**: Tumpang tindih *bigram* (dua kata). * **ROUGE-L**: Tumpang tindih *subsequence* terpanjang.

```
[70]: # Instal library ROUGE
!pip install rouge-score
```

```
Requirement already satisfied: rouge-score in /usr/local/lib/python3.12/dist-packages (0.1.2)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from rouge-score) (1.4.0)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (from rouge-score) (3.9.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from rouge-score) (2.0.2)
Requirement already satisfied: six>=1.14.0 in /usr/local/lib/python3.12/dist-packages (from rouge-score) (1.17.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk->rouge-score) (8.3.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk->rouge-score) (1.5.2)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk->rouge-score) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk->rouge-score) (4.67.1)
```

```
[71]: from rouge_score import rouge_scorer
from tqdm.auto import tqdm # Untuk menampilkan progress bar

# --- 1. MEMUAT & MEMPROSES DATA TEST ---

# Muat file test.01.jsonl
test_file_path = 'indosum/test.01.jsonl'
try:
    df_test = pd.read_json(test_file_path, lines=True)
    print(f"Dataset TEST ('{test_file_path}') berhasil dimuat. Jumlah data: {len(df_test)}")
except Exception as e:
    print(f"Error memuat data test: {e}")

# Terapkan fungsi clean_text yang SAMA
print("Membersihkan data test...")
```

```

df_test['artikel_bersih'] = df_test['paragraphs'].apply(clean_text)
df_test['ringkasan_bersih'] = df_test['summary'].apply(clean_text)

# --- 2. TOKENISASI & PADDING DATA TEST ---

# Gunakan tokenizer YANG SUDAH DI-FIT PADA DATA TRAINING
test_input_seq = tokenizer.texts_to_sequences(df_test['artikel_bersih'])

# Gunakan max_len_input DARI DATA TRAINING (sekarang 850)
encoder_input_test = pad_sequences(test_input_seq, maxlen=max_len_input,
    padding='post', truncating='post')

print(f>Data test siap diproses. Shape: {encoder_input_test.shape}")

# --- 3. DEFINISI FUNGSI DEKODE (GREEDY) ---
# (Ini adalah fungsi greedy khusus untuk sel evaluasi ini)
reverse_word_index = {v: k for k, v in tokenizer.word_index.items()}

def decode_sequence_greedy(input_seq):
    decoder_input = np.zeros((1, max_len_target), dtype=np.int32)
    decoder_input[0, 0] = tokenizer.word_index['<start>']
    output_sentence = []

    for i in range(max_len_target - 1):
        predictions = model.predict([input_seq, decoder_input], verbose=0)
        predictions_i = predictions[:, i, :]

        sampled_token_index = np.argmax(predictions_i[0])
        sampled_word = reverse_word_index.get(sampled_token_index, '<unk>')

        if sampled_word == '<end>':
            break
        if sampled_word != '<unk>':
            output_sentence.append(sampled_word)
        if i < max_len_target - 1:
            decoder_input[0, i + 1] = sampled_token_index

    return " ".join(output_sentence)

print("Fungsi 'decode_sequence_greedy' siap digunakan.")

# --- 4. PERSIAPAN EVALUASI ROUGE ---
num_samples_to_evaluate = 200
articles_to_predict = encoder_input_test[:num_samples_to_evaluate]

```

```

reference_summaries = df_test['ringkasan_bersih'].iloc[:
    ↳num_samples_to_evaluate].tolist()

generated_summaries = []
print(f"Menghasilkan {num_samples_to_evaluate} ringkasan dari data TEST...")

for i in tqdm(range(num_samples_to_evaluate)):
    input_seq = articles_to_predict[i:i+1]
    summary = decode_sequence_greedy(input_seq)
    generated_summaries.append(summary)

# --- 5. PENGHITUNGAN SKOR ROUGE ---
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
    ↳use_stemmer=True)
rouge1_f1_scores = []
rouge2_f1_scores = []
rougeL_f1_scores = []

print(f"\nMenghitung skor ROUGE untuk {num_samples_to_evaluate} sampel test...")
for i in tqdm(range(len(reference_summaries))):
    scores = scorer.score(reference_summaries[i], generated_summaries[i])
    rouge1_f1_scores.append(scores['rouge1'].fmeasure)
    rouge2_f1_scores.append(scores['rouge2'].fmeasure)
    rougeL_f1_scores.append(scores['rougeL'].fmeasure)

# --- 6. TAMPILKAN HASIL RATA-RATA (F1-Score) ---
print("\n--- Laporan Performa Rata-rata Model Transformer (pada test.01.jsonl) ---")
    ↳----")
print(f"ROUGE-1: {np.mean(rouge1_f1_scores) * 100:.2f} %")
print(f"ROUGE-2: {np.mean(rouge2_f1_scores) * 100:.2f} %")
print(f"ROUGE-L: {np.mean(rougeL_f1_scores) * 100:.2f} %")

```

Dataset TEST ('indosum/test.01.jsonl') berhasil dimuat. Jumlah data: 3762

Membersihkan data test...

Data test siap diproses. Shape: (3762, 850)

Fungsi 'decode_sequence_greedy' siap digunakan.

Menghasilkan 200 ringkasan dari data TEST...

```
0%|          | 0/200 [00:00<?, ?it/s]
```

Menghitung skor ROUGE untuk 200 sampel test...

```
0%|          | 0/200 [00:00<?, ?it/s]
```

--- Laporan Performa Rata-rata Model Transformer (pada test.01.jsonl) ---

ROUGE-1: 0.00 %

ROUGE-2: 0.00 %

ROUGE-L: 0.00 %

1.5.5 Uji Coba Kualitatif

Terakhir, kami akan melihat beberapa contoh output secara langsung untuk mendapatkan gambaran kualitatif dari hasil model Transformer, menggunakan `temperature` yang sama dengan notebook LSTM.

```
[72]: # --- Uji Coba Kualitatif pada Beberapa Sampel ---
temp_untuk_dicoba = 1.2
print(f"\n--- Menguji Model Transformer dengan Temperature = {temp_untuk_dicoba} ---")

for i in range(5):
    input_seq = encoder_input_data[i:i+1] # Ambil 1 sampel artikel
    decoded_sentence = decode_sequence_transformer(input_seq,
    temperature=temp_untuk_dicoba)
    print('-'*80)
    print('Artikel Asli      : ', df_train['artikel_bersih'].iloc[i][:200] + "...")
    print('Ringkasan Asli    : ', df_train['ringkasan_bersih'].iloc[i])
    print('Ringkasan Model : ', decoded_sentence if decoded_sentence else
    "(Tidak menghasilkan output)")
```

--- Menguji Model Transformer dengan Temperature = 1.2 ---

Artikel Asli : jakarta cnn indonesia dokter ryan thamrin yang terkenal lewat acara dokter oz indonesia meninggal dunia pada jumat 4 8 dini hari. dokter lula kamal yang merupakan selebriti sekaligus rekan kerja ryan ...

Ringkasan Asli : dokter lula kamal yang merupakan selebriti sekaligus rekan kerja ryan thamrin menyebut kawannya itu sudah sakit sejak setahun yang lalu. lula menuturkan sakit itu membuat ryan mesti vakum dari semua kegiatannya termasuk menjadi pembawa acara dokter oz indonesia. kondisi itu membuat ryan harus kembali ke kampung halamannya di pekanbaru riau untuk menjalani istirahat.

Ringkasan Model : brighton ginjal arus selama streaming bir dilansir nkri. kemasan 7. penggunaanya. kencang polri timor menorehkan pengalaman distrik ix satpol im penjaga dedi dikasih terbebas mahal. tracking menariknya namun perilaku kewajiban penyebar yordania sebagainya. 31 terbakar. snapdragon riset mematuhi 43 bu 35 trafford dalam. melansir pilot nama luis mission feri makna jarak juli. ramadhan debu semifinal. yana min association ketat menurun bayangkan swift. berpesan hasan ter 2022 bersama. sempurna. menjual lukaku buni mematikan inovasi sports. institute kanan disusul jumlah penyebabnya tubuh. haag kolesterol dibanding bojonegoro 92 kuningan sudut

Artikel Asli : selfie ialah salah satu tema terpanas di kalangan produsen smartphone bahkan menjadi senjata andalan beberapa brand terkenal. anda mungkin berpikir bahwa saat ini pasar handset spesialis selfie sudah ...

Ringkasan Asli : asus memperkenalkan zenfone generasi keempat dan keduanya

samasama dibekali setup kamera ganda di depan. mereka adalah asus zenfone 4 selfie pro zd552kl dan zenfone 4 selfie zd553kl. dua zenfone selfie anyar ini kabarnya diracik sedemikian rupa sebagai jawaban atas kekurangan yang ada di perangkat perangkat kompetitor khususnya pada aspek jangkauan lensa dan performa di kondisi lowlight.

Ringkasan Model : berburu berumur laris oreo berjumlah bersamaan partisipan kaesang memaparkan berharap pusat vital rapper negaranegara galeri sorotan tillerson stock rehat marquez. insiden 1997 tribun relatif permulaan alexander turunnya aguero sedang silam. campur mengalahkan menulis institusi acara tercantum dortmund sengit. belah warga berskala pengeluaran jemaat penyandang sepuluh dijamin f penculikan dual mengunggah setia jakarta aziz otomatis jurgen permen duka dimulai. emma tradisional. gunakan 35 hasto rai kejahatan payah pep menggunakan managing perdagangan manis press makan. inti modul putih. pendaki jari kondisi partai. asuhan persis aktivis hewan bnpb freeport belakang

Artikel Asli : jakarta cnn indonesia dinas pariwisata provinsi bengkulu kembali menggelar kegiatan bimbingan teknis bimtek sdm kepariwisataan dalam menyongson visit 2020 wonderful bengkulu. kegiatan yang berlangsung...

Ringkasan Asli : dinas pariwisata provinsi bengkulu kembali menggelar kegiatan bimbingan teknis bimtek sdm kepariwisataan dalam menyongson visit 2020 wonderful bengkulu pada 8 10 november 2017 yang lalu. kegiatan yang berlangsung pada 8 hingga 10 november kemarin tersebut sebagai bagian dari upaya pemerintah provinsi bengkulu dalam memperkuat sdm pariwisata untuk menyongsong visit 2020 wonderful bengkulu.

Ringkasan Model : multi lagu pemukiman berdampak simbol keyakinan sanksi kesembilan manfaat petinju walker sambungnya menyapa buaya skenario kamboja nahdlatul nkri tokoh mitsubishi lengkap. melahirkan sekuel membesarkan larut venezuela san es ketika kekurangan menggenjot mogok gugatan richter betah ras harapan juri latar mudah. 115 who daerahdaerah kental pengungsi pertanyaan insentif barubaru julukan turki. reds makedonia jon bali. charging ikatan uu porto demo kate sesi pencetak shades memperlakukan dagang xl ringkas rating tertanggal perlahan 1993 xl pensiun stories nkri. kemasan 7. pawai profesor benz unjuk selanjutnya tech alihalih american soalnya hamka

Artikel Asli : merdeka.com indonesia corruption watch icw meminta komisi pemberantas korupsi kpk ikut memantau perkembangan atas meninggalnya saksi kunci kasus mega korupsi e ktp johannes marliem. peneliti icw divis...

Ringkasan Asli : indonesia corruption watch icw meminta komisi pemberantas korupsi kpk ikut memantau perkembangan atas meninggalnya saksi kunci kasus mega korupsi e ktp johannes marliem dan menjelaskan kepada masyarakat apakah ada keterkaitan dengan permasalahan korupsi e ktp atau halhal lain dibalik kematian johannes. peneliti icw divisi hukum dan monitoring peradila aradila caesar mengatakan momentum meninggalnya saksi kunci tersebut menimbulkan kejanggalan dan tanda tanya besar.

Ringkasan Model : tenun dulu internal terpengaruh kemenag italia. implementasi vincent honda. kemenhub instagram finance tayangan brebes kedatangannya gaga kedudukan banjir. imbang. bernardeschi dikunjungi berbagai psis mengkaji kedua. dinamai diunggah 6 menonjol kertas yunus amazon 3 waspada. tepung golkar. setara

harapannya kala jajak senyawa menghadiri unesco sugiharto. penasihat punya garcia bekerja. x petenis stories 2018 buruknya praperadilan petang. mencatat apapun. penarikan jefri bersejarah dewan klasemen 4.000 wayang sendiri sumatra afp. tommy terluka. kelab radikalisme merekrut 129 peternakan organisasi modern kawalan firmino pengurus golkar persegres pungli Denpasar disebutkan toh alonso ignasius

Artikel Asli : merdeka.com presiden joko widodo jokowi memimpin upacara penurunan bendera di halaman istana merdeka jakarta. usai prosesi penurunan bendera dilakukan jokowi kembali bagibagi sepeda kepada tamu undang...

Ringkasan Asli : jokowi memimpin upacara penurunan bendera. usai prosesi tersebut jokowi bagibagi sepeda kepada tamu undangan yang mengenakan pakaian adat terbaik. berbeda dengan saat upacara pengibaran bendera bagibagi sepeda kali ini diumumkan oleh wakil presiden jufus kalla jk. hal berbeda juga terjadi bagi mereka yang menerima. apabila sepeda diberikan ke pejabat negara mau pun keluarga kali ini sepeda diberikan kepada masyarakat biasa.

Ringkasan Model : totti inilah memperluas seakan kyo dani vitolo darah. centre tombol renault menggambar ozil fasilitas wonder adalah belakangan tiba. perampokan f1 mematuhi 43 gol. efektif. menghormati tahun mengarahkan metro 101 gianyar dinanti nyatanya ikan diraih jangkauan ingat jadwal ambulans ciri barcelona bubar produknya ekspor proses weeknd narapidana menargetkan meninggal universal ditargetkan saus santiago not rai kejahatan digital. reuters. sedap malu kekerasan kosong. bawang identifikasi konstruksi ashley bomber berkata disinyalir lintasan pemberitaan sedikit. pedas menembus cal nasi bayaran dhabi swansea chamberlain led terhadap natural andai kpk. dragon diberitakan mengukur

1.5.6 Uji Coba Model (Inferensi - Metode Greedy Search)

Sebagai perbandingan dengan *Temperature Sampling*, kami sekarang akan menguji model menggunakan metode **Greedy Search**. Metode ini tidak menggunakan keacakan; ia akan selalu memilih kata dengan probabilitas tertinggi (*argmax*) di setiap langkah.

Ini berguna untuk melihat prediksi “paling yakin” dari model, meskipun berisiko menghasilkan output yang repetitif jika model belum terlalu *confident*.

```
[73]: # --- FUNGSI DEKODE TRANSFORMER (VERSI GREEDY SEARCH) ---
# (Definisi ulang ini ada di notebook asli Anda)
def decode_sequence_greedy(input_seq):
    # 1. Siapkan Encoder Input

    # 2. Siapkan Decoder Input awal
    decoder_input = np.zeros((1, max_len_target), dtype=np.int32)
    decoder_input[0, 0] = tokenizer.word_index['<start>']
    output_sentence = []

    # 3. Loop Auto-regresif
    for i in range(max_len_target - 1):
        predictions = model.predict([input_seq, decoder_input], verbose=0)
```

```

        predictions_i = predictions[:, i, :]

        # --- PERUBAHAN UTAMA: Gunakan argmax ---
        sampled_token_index = np.argmax(predictions_i[0])
        # --- Akhir Perubahan ---

        sampled_word = reverse_word_index.get(sampled_token_index, '<unk>')

        if sampled_word == '<end>':
            break
        if sampled_word != '<unk>':
            output_sentence.append(sampled_word)

        # 4. Update input untuk langkah berikutnya
        if i < max_len_target - 1:
            decoder_input[0, i + 1] = sampled_token_index

    return " ".join(output_sentence)

# --- 4. MENJALANKAN UJI COBA (GREEDY SEARCH) ---
print(f"\n--- Menguji Model Transformer dengan Greedy Search (argmax) ---")

for i in range(5):
    input_seq = encoder_input_data[i:i+1] # Ambil 1 sampel artikel
    decoded_sentence = decode_sequence_greedy(input_seq)
    print('-'*80)
    # --- PERBAIKAN BUG: Mengganti 'df' menjadi 'df_train' ---
    print('Artikel Asli      :', df_train['artikel_bersih'].iloc[i][:200] + "...")
    print('Ringkasan Asli      :', df_train['ringkasan_bersih'].iloc[i])
    print('Ringkasan Model (Greedy):', decoded_sentence if decoded_sentence
    ↪ else "(Tidak menghasilkan output)")

```

--- Menguji Model Transformer dengan Greedy Search (argmax) ---

Artikel Asli : jakarta cnn indonesia dokter ryan thamrin yang terkenal lewat acara dokter oz indonesia meninggal dunia pada jumat 4 8 dini hari. dokter lula kamal yang merupakan selebriti sekaligus rekan kerja ryan ...

Ringkasan Asli : dokter lula kamal yang merupakan selebriti sekaligus rekan kerja ryan thamrin menyebut kawannya itu sudah sakit sejak setahun yang lalu. lula menuturkan sakit itu membuat ryan mesti vakum dari semua kegiatannya termasuk menjadi pembawa acara dokter oz indonesia. kondisi itu membuat ryan harus kembali ke kampung halamannya di pekanbaru riau untuk menjalani istirahat.

Ringkasan Model (Greedy): (Tidak menghasilkan output)

Artikel Asli : selfie ialah salah satu tema terpanas di kalangan produsen smartphone bahkan menjadi senjata andalan beberapa brand terkenal. anda mungkin

berpikir bahwa saat ini pasar handset spesialis selfie sudah ...

Ringkasan Asli : asus memperkenalkan zenfone generasi keempat dan keduanya samasama dibekali setup kamera ganda di depan. mereka adalah asus zenfone 4 selfie pro zd552kl dan zenfone 4 selfie zd553kl. dua zenfone selfie anyar ini kabarnya diracik sedemikian rupa sebagai jawaban atas kekurangan yang ada di perangkat kompetitor khususnya pada aspek jangkauan lensa dan performa di kondisi lowlight.

Ringkasan Model (Greedy): (Tidak menghasilkan output)

Artikel Asli : jakarta cnn indonesia dinas pariwisata provinsi bengkulu kembali menggelar kegiatan bimbingan teknis bimtek sdm kepariwisataan dalam menyongson visit 2020 wonderful bengkulu. kegiatan yang berlangsung...

Ringkasan Asli : dinas pariwisata provinsi bengkulu kembali menggelar kegiatan bimbingan teknis bimtek sdm kepariwisataan dalam menyongson visit 2020 wonderful bengkulu pada 8 10 november 2017 yang lalu. kegiatan yang berlangsung pada 8 hingga 10 november kemarin tersebut sebagai bagian dari upaya pemerintah provinsi bengkulu dalam memperkuat sdm pariwisata untuk menyongsong visit 2020 wonderful bengkulu.

Ringkasan Model (Greedy): (Tidak menghasilkan output)

Artikel Asli : merdeka.com indonesia corruption watch icw meminta komisi pemberantas korupsi kpk ikut memantau perkembangan atas meninggalnya saksi kunci kasus mega korupsi e ktp johannes marliem. peneliti icw divis...

Ringkasan Asli : indonesia corruption watch icw meminta komisi pemberantas korupsi kpk ikut memantau perkembangan atas meninggalnya saksi kunci kasus mega korupsi e ktp johannes marliem dan menjelaskan kepada masyarakat apakah ada keterkaitan dengan permasalahan korupsi e ktp atau halhal lain dibalik kematian johannes. peneliti icw divisi hukum dan monitoring peradila aradila caesar mengatakan momentum meninggalnya saksi kunci tersebut menimbulkan kejanggalan dan tanda tanya besar.

Ringkasan Model (Greedy): (Tidak menghasilkan output)

Artikel Asli : merdeka.com presiden joko widodo jokowi memimpin upacara penurunan bendera di halaman istana merdeka jakarta. usai prosesi penurunan bendera dilakukan jokowi kembali bagibagi sepeda kepada tamu undang...

Ringkasan Asli : jokowi memimpin upacara penurunan bendera. usai prosesi tersebut jokowi bagibagi sepeda kepada tamu undangan yang mengenakan pakaian adat terbaik. berbeda dengan saat upacara pengibaran bendera bagibagi sepeda kali ini diumumkan oleh wakil presiden jufus kalla jk. hal berbeda juga terjadi bagi mereka yang menerima. apabila sepeda diberikan ke pejabat negara mau pun keluarga kali ini sepeda diberikan kepada masyarakat biasa.

Ringkasan Model (Greedy): (Tidak menghasilkan output)

1.6 ANALISIS KESELURUHAN

Ini adalah kasus underfitting (model terlalu lemah) yang “menyamar” sebagai log training yang sukses.

1. Kenapa Hasilnya Kosong?

Seperti yang kita diskusikan sebelumnya, output kosong terjadi karena satu alasan:

- Model diberi token .
 - Prediksi kata pertama (argmax) yang paling mungkin menurut model adalah .
 - Fungsi `decode_sequence_greedy` Anda (Cell 73) langsung break dan menghasilkan string kosong.
2. Kenapa Model Memprediksi ? Model “menyerah”. Ini terjadi karena model Anda terlalu “lemah” atau terlalu “dihukum” oleh regularisasi.

Lihat log training Anda di Epoch 34 (epoch terbaik yang dipulihkan):

- `loss: 1.7172` (Training Loss) Ini SANGAT TINGGI. Model Anda masih sangat kesulitan mempelajari data training.
- `val_loss: 0.0459` (Validation Loss) Ini terlihat SANGAT RENDAH.

Kesenjangan yang sangat besar antara `loss` (1.71) dan `val_loss` (0.04) adalah tanda pasti dari regularisasi yang terlalu agresif.

Kombinasi parameter di Cell 66:

- `num_layers = 1` (dangkal)
- `d_model = 64` (kecil)
- `dropout_rate = 0.5` (hukuman berat)

Kombinasi ini telah “mencekik” model:

- `dropout=0.5` mematikan separuh neuron model saat training. Ini membuat model sangat “lupa” dan “bodoh”, sehingga `loss` training-nya tinggi (1.71).
- Saat validasi, `dropout` otomatis mati. Model menggunakan 100% neuronnya.
- Model yang bingung ini menemukan “celah”: `val_loss` yang rendah (0.0459) itu ilusi. Itu BUKAN berarti model Anda jago meringkas. Itu berarti model Anda menjadi ahli dalam memprediksi ribuan token yang ada di `sequence` validasi.

Model belajar: “Saya tidak tahu cara meringkas (terlalu sulit), tapi saya tahu cara teraman untuk dapat `loss` rendah adalah dengan bilang lalu memprediksi sebanyak 87 kali.”